Typescript Fundamentals

4A. TypeScript Fundamentals

Questions: 8 **Difficulty:** Junior to Mid **Companies:** Asked at nearly every product company — Google, Amazon, Flipkart, Razorpay, Swiggy, Atlassian, Microsoft, Cred

---

Q 4.1 Why TypeScript? What problems does it solve and when does it add unnecessary overhead?

Difficulty: Junior **Asked at:** Razorpay, Flipkart, Swiggy, Walmart

This sounds like a softball question but interviewers use it to filter out people who give a rehearsed answer like "TypeScript adds types to JavaScript." They want to hear you talk about real problems you've hit.

JavaScript doesn't tell you when things go wrong until runtime. You rename a property in an API response, and the code doesn't break during development — it breaks in production when a user clicks a button and `undefined.map()` throws. TypeScript catches that at compile time, before the code ever runs.

But it goes beyond catching typos. TypeScript acts as living documentation. When you open a function and see this:

TYPESCRIPT Copy

```
function createOrder(
  items: CartItem[],
  customer: Customer,
```

You immediately know what this function expects, what's optional, and what it returns. No need to read the implementation, search for usage examples, or check documentation that might be outdated.

The third real benefit is refactoring confidence. When you change a type definition — say you rename `Customer.name` to `Customer.fullName` — TypeScript shows you every single place in the codebase that references the old property. In a JavaScript codebase, you'd use find-and-replace and pray you didn't miss one.

When TypeScript adds overhead:

Small scripts, quick prototypes, or one-off utilities where the code will be written once and rarely touched. If you're writing a 50-line build script, adding TypeScript configuration, type definitions, and compilation steps is overkill. The type system has a learning curve for generics and advanced patterns that can slow down a team if they're new to it.

The honest answer is: for any frontend project that multiple people work on or that will be maintained beyond a few months, TypeScript pays for itself. For throwaway scripts and rapid prototyping, plain JavaScript is faster to start with.

Follow-up interviewers ask: "Have you ever introduced TypeScript to a project? How did the team react?"

This is where they want a real story. Talk about the migration approach (incremental, `allowJs: true`), the initial resistance ("it slows us down"), and the turning point (someone caught a production bug at compile time).

Q 4.2 How does type inference work? When should you explicitly annotate vs let TypeScript infer?

Difficulty: Junior **Asked at:** Amazon, Google, Atlassian

TypeScript can figure out types automatically in most situations. You don't need to write them all out. This is type inference.

TYPESCRIPT Copy

```typescript
// TypeScript infers these automatically — no annotation needed
const name = "Ankit";           // inferred as string
const age = 28;                 // inferred as number
const isActive = true;          // inferred as boolean
const scores = [90, 85, 72];    // inferred as number[]

// Function return type is inferred from the return statement
function double(n: number) {
  return n * 2;  // return type inferred as number
}

// Object shape is inferred from the structure
const user = {
  id: 1,
  name: "Ankit",
  role: "admin" as const,  // inferred as "admin", not string
};
```

When to let TypeScript infer:

- Variable declarations with initial values — `const x = 5` is clearer than `const x: number = 5`
- Function return types when the return value is obvious
- Array methods where types flow through — `items.map(item => item.name)` infers correctly
- Destructuring — `const { name, age } = user` inherits the types

When to annotate explicitly:

TYPESCRIPT Copy

```typescript
// 1. Function parameters — TypeScript can't infer these
function greet(name: string, age: number): void {
  console.log(`Hello ${name}, you are ${age}`);
```

```
// 2. Empty arrays — TypeScript infers never[] without annotation
const items: string[] = [];

// 3. Complex return types — makes the contract clear
function fetchUser(id: string): Promise<User | null> {
  // ...
}

// 4. When inference gives you a wider type than you want
let status: "loading" | "success" | "error" = "loading";
// Without annotation, TypeScript infers 'string' — too wide

// 5. Object declarations without initial values
let config: AppConfig;
```

The rule of thumb: annotate function signatures (parameters and return types for public APIs), let inference handle everything inside the function body. This gives you the safety of explicit types at boundaries while keeping internal code clean and readable.

Common mistake: Over-annotating. When you write `const x: number = 5`, the `: number` is noise. TypeScript already knows it's a number. Over-annotation makes code harder to read and harder to refactor because you're maintaining types in two places — the value and the annotation.

Q 4.3 Explain primitive types, literal types, union types, and intersection types.

Difficulty: Junior **Asked at:** Flipkart, Swiggy, PhonePe, Cred

These are the building blocks. Everything else in TypeScript's type system is built from these.

Primitive types — the basic value types:

TYPESCRIPT Copy

```
let isActive: boolean = true;
let id: bigint = 100n;
let unique: symbol = Symbol("id");
let nothing: null = null;
let missing: undefined = undefined;
```

Literal types — a specific value, not just a category:

TYPESCRIPT Copy

```
// 'string' means any string. "admin" means exactly the string "admin"
let role: "admin" | "editor" | "viewer" = "admin";

// Same with numbers and booleans
let statusCode: 200 | 404 | 500 = 200;
let enabled: true = true;
```

Literal types become powerful when combined with unions because they restrict values to an exact set of options. This is how you model things like status codes, user roles, and API action types.

Union types — a value that can be one of several types:

TYPESCRIPT Copy

```
// This value is either a string OR a number
type ID = string | number;

function findUser(id: ID) {
  if (typeof id === "string") {
    // TypeScript knows id is string here
    return db.findBySlug(id.toLowerCase());
  }
  // TypeScript knows id is number here
  return db.findById(id);
}

// Discriminated union — the most useful pattern in TypeScript
```

```
  | { status: "loading" };

function handleResponse(response: ApiResponse) {
  switch (response.status) {
    case "success":
      // TypeScript knows response.data exists here
      console.log(response.data.name);
      break;
    case "error":
      // TypeScript knows response.message exists here
      console.log(response.message);
      break;
    case "loading":
      // Neither data nor message is available
      break;
  }
}
```

Intersection types — a value that is ALL of the listed types combined:

TYPESCRIPT Copy

```
type HasName = { name: string };
type HasAge = { age: number };
type HasEmail = { email: string };

// Person must have ALL three properties
type Person = HasName & HasAge & HasEmail;

const person: Person = {
  name: "Ankit",
  age: 28,
  email: "ankit@example.com",
};

// Practical use: extending a base type
type BaseEntity = { id: string; createdAt: Date };
```

The mental model: Union is "OR" — the value is one of these types. Intersection is "AND" — the value has all of these types combined.

Follow-up: "When would you use a union type vs an enum?"

Unions are preferred in 2026. Enums generate runtime JavaScript code (an object with reverse mappings), increasing bundle size. String unions (`"admin" | "editor"`) are purely a compile-time construct — they disappear completely after compilation. Unions also work better with type narrowing and are more idiomatic in React codebases.

Q 4.4 Type aliases vs interfaces — when do you use which and why do both exist?

Difficulty: Junior **Asked at:** Google, Amazon, Flipkart, Microsoft

Both let you name a type. For most everyday use, they're interchangeable:

TYPESCRIPT Copy

```typescript
// These are functionally identical for object shapes
type UserType = {
  name: string;
  age: number;
};

interface UserInterface {
  name: string;
  age: number;
}

// Both work the same way
const user1: UserType = { name: "Ankit", age: 28 };
const user2: UserInterface = { name: "Ankit", age: 28 };
```

But there are real differences that matter in practice.

Only `type` can do these:

TYPESCRIPT Copy

```typescript
// Union types
type Status = "loading" | "success" | "error";

// Primitive aliases
type ID = string | number;

// Tuple types
type Coordinate = [number, number];

// Mapped types and conditional types
type Readonly<T> = { readonly [K in keyof T]: T[K] };
```

Only `interface` can do these:

TYPESCRIPT Copy

```typescript
// Declaration merging — extending an existing interface
interface Window {
  analytics: AnalyticsSDK;
}
// Now window.analytics is typed, merged with the built-in Window interfac

// Extending other interfaces
interface Animal {
  name: string;
}

interface Dog extends Animal {
  breed: string;
}
```

Declaration merging is the key differentiator. When you declare an interface with the same name twice, TypeScript merges them. This is how library type definitions work —

you can extend `Window`, `HTMLElement`, or any third-party library's types by declaring an interface with the same name.

The practical rule most teams follow:

- Use `interface` for object shapes that might be extended — component props, API contracts, public library types.
- Use `type` for everything else — unions, intersections, tuples, mapped types, utility compositions.

TYPESCRIPT Copy

```typescript
// Interface for component props (might be extended)
interface ButtonProps {
  label: string;
  onClick: () => void;
  variant?: "primary" | "secondary";
}

// Type for unions, computed types, internal types
type ButtonVariant = "primary" | "secondary" | "danger";
type PartialButtonProps = Partial<ButtonProps>;
type ButtonOrLink = ButtonProps | AnchorProps;
```

What interviewers don't want to hear: "I always use interface" or "I always use type." They want to know you understand the difference and make intentional choices.

Q 4.5 Why are string literal unions preferred over enums in 2026? What is `as const`?

Difficulty: Mid **Asked at:** Razorpay, Cred, Google, Swiggy

Enums have been a TypeScript feature since the beginning, but the community has largely moved away from them. Here's why:

Problem 1: Enums generate runtime code

```typescript
// This TypeScript
enum Status {
  Loading = "loading",
  Success = "success",
  Error = "error",
}

// Compiles to this JavaScript — actual code that ships to users
var Status;
(function (Status) {
  Status["Loading"] = "loading";
  Status["Success"] = "success";
  Status["Error"] = "error";
})(Status || (Status = {}));
```

A string union compiles to nothing:

TYPESCRIPT Copy

```typescript
// This TypeScript
type Status = "loading" | "success" | "error";

// Compiles to... nothing. Zero runtime bytes.
```

Problem 2: Numeric enums aren't type-safe

TYPESCRIPT Copy

```typescript
enum Direction {
  Up,    // 0
  Down,  // 1
  Left,  // 2
  Right, // 3
}

// TypeScript allows ANY number — this should be an error but isn't
const d: Direction = 42;  // No error!
```

Problem 3: Enums don't play well with union type narrowing

String unions work seamlessly with `switch`, `if`, and discriminated unions. Enums require importing the enum at every usage site, adding coupling between files.

TYPESCRIPT Copy

```typescript
// String union — simple, no imports needed
type Theme = "light" | "dark" | "system";

function applyTheme(theme: Theme) {
  switch (theme) {
    case "light": /* ... */ break;
    case "dark": /* ... */ break;
    case "system": /* ... */ break;
  }
}

// vs Enum — requires importing the enum everywhere
import { Theme } from "./types";
applyTheme(Theme.Light);
```

`as const` — turning objects into type-safe constants:

TYPESCRIPT Copy

```typescript
// Without as const — types are widened
const config = {
  api: "https://api.example.com",
  timeout: 5000,
  retries: 3,
};
// typeof config = { api: string; timeout: number; retries: number }

// With as const — types are narrowed to exact values
const config = {
  api: "https://api.example.com",
  timeout: 5000,
  retries: 3,
```

`as const` makes every property `readonly` and narrows the type to the exact literal value. This is the modern replacement for enums:

TYPESCRIPT Copy

```typescript
const STATUS = {
  Loading: "loading",
  Success: "success",
  Error: "error",
} as const;

// Extract the union type from the object values
type Status = typeof STATUS[keyof typeof STATUS];
// Result: "loading" | "success" | "error"
```

You get the best of both worlds — a named object you can reference (`STATUS.Loading`), and a type-safe union that compiles to zero runtime code.

Q 4.6 How does type narrowing work? Explain the different techniques.

Difficulty: Mid **Asked at:** Google, Amazon, Atlassian, Microsoft, Flipkart

Type narrowing is TypeScript's ability to figure out a more specific type within a conditional block. You start with a wide type (like `string | number`) and narrow it down to a specific type (like `string`) based on checks you make in code.

TypeScript understands several narrowing techniques:

`typeof` checks — for primitives

TYPESCRIPT Copy

```
    // TypeScript knows value is string here
    return value.toUpperCase();
  }
  // TypeScript knows value is number here
  return value.toFixed(2);
}
```

`instanceof` checks — for class instances

TYPESCRIPT Copy

```
function getArea(shape: Circle | Rectangle): number {
  if (shape instanceof Circle) {
    return Math.PI * shape.radius ** 2;
  }
  // TypeScript knows shape is Rectangle here
  return shape.width * shape.height;
}
```

`in` operator — checking if a property exists

TYPESCRIPT Copy

```
type Fish = { swim: () => void };
type Bird = { fly: () => void };

function move(animal: Fish | Bird) {
  if ("swim" in animal) {
    animal.swim();  // TypeScript knows it's Fish
  } else {
    animal.fly();   // TypeScript knows it's Bird
  }
}
```

Discriminated unions — the most powerful pattern

TYPESCRIPT Copy

```
  | { kind: "rectangle"; width: number; height: number }
  | { kind: "triangle"; base: number; height: number };

function getArea(shape: Shape): number {
  switch (shape.kind) {
    case "circle":
      return Math.PI * shape.radius ** 2;
    case "rectangle":
      return shape.width * shape.height;
    case "triangle":
      return 0.5 * shape.base * shape.height;
  }
}
```

The `kind` property is the discriminant — a common property with literal types that TypeScript uses to narrow the union.

User-defined type guards — custom narrowing functions

TYPESCRIPT Copy

```
type User = { type: "user"; name: string };
type Admin = { type: "admin"; name: string; permissions: string[] };

// The `is` keyword tells TypeScript this function narrows the type
function isAdmin(person: User | Admin): person is Admin {
  return person.type === "admin";
}

function greet(person: User | Admin) {
  if (isAdmin(person)) {
    // TypeScript knows person is Admin here
    console.log(`Admin ${person.name} has ${person.permissions.length} per
  } else {
    console.log(`User ${person.name}`);
  }
}
```

TYPESCRIPT Copy

```typescript
function printName(name: string | null | undefined) {
  if (name) {
    // TypeScript knows name is string here (truthy)
    console.log(name.toUpperCase());
  }
}

// Also works with optional chaining
const length = name?.length;  // number | undefined
```

Follow-up: "What is the exhaustive check pattern?"

Add a `never` check in the default case. If you add a new variant to the union and forget to handle it, TypeScript throws a compile error:

TYPESCRIPT Copy

```typescript
function getArea(shape: Shape): number {
  switch (shape.kind) {
    case "circle": return Math.PI * shape.radius ** 2;
    case "rectangle": return shape.width * shape.height;
    case "triangle": return 0.5 * shape.base * shape.height;
    default:
      // If you add a new shape kind and don't handle it above,
      // TypeScript errors here because shape is not 'never'
      const _exhaustive: never = shape;
      throw new Error(`Unhandled shape: ${_exhaustive}`);
  }
}
```

Q 4.7 Explain strict null checks, optional chaining, and nullish coalescing.

Difficulty: Junior **Asked at:** Flipkart, Swiggy, PhonePe, Walmart

These three features work together to handle the most common source of runtime errors in JavaScript — accessing properties on `null` or `undefined`.

Strict null checks (`strictNullChecks: true` in tsconfig)

Without strict null checks, TypeScript allows `null` and `undefined` to be assigned to any type. This is the same footgun as plain JavaScript:

TYPESCRIPT Copy

```
// strictNullChecks: false — TypeScript allows this (dangerous)
let name: string = null;      // No error
name.toUpperCase();           // Runtime crash: Cannot read property of nul

// strictNullChecks: true — TypeScript catches this (safe)
let name: string = null;      // Error: Type 'null' is not assignable to ty
let name: string | null = null;  // You must explicitly opt into null
```

With strict null checks enabled, you have to explicitly handle the `null` case before accessing properties:

TYPESCRIPT Copy

```
function getLength(name: string | null): number {
  // Error: name might be null
  return name.length;

  // Correct: check first
  if (name === null) return 0;
  return name.length;  // TypeScript knows name is string here
}
```

Optional chaining (`?.`)

Safely access deeply nested properties without checking every level:

TYPESCRIPT Copy

```
// Without optional chaining — verbose and painful
const city = user && user.address && user.address.city;
```

```
// Returns undefined if any part of the chain is null/undefined

// Works with method calls
const result = api?.getData?.();

// Works with array access
const first = items?.[0];
```

If any part of the chain is `null` or `undefined`, the expression short-circuits and returns `undefined` instead of throwing.

Nullish coalescing (`??`)

Provides a fallback value when something is `null` or `undefined` — but NOT for other falsy values like `0`, `""`, or `false`.

TYPESCRIPT Copy

```
// The problem with || (logical OR)
const port = config.port || 3000;
// If config.port is 0, this returns 3000 (wrong! 0 is a valid port)

// ?? only falls back for null/undefined
const port = config.port ?? 3000;
// If config.port is 0, this returns 0 (correct)
// If config.port is null or undefined, this returns 3000

// More examples where ?? beats ||
const count = response.count ?? 0;    // count of 0 is valid
const name = input.name ?? "Anonymous"; // empty string "" is a valid name
const enabled = settings.enabled ?? true; // false is a valid value
```

All three together in practice:

TYPESCRIPT Copy

```
function getUserDisplay(user: User | null): string {
  // Optional chaining to safely access nested properties
```

```
  const role = user?.permissions?.role ?? "viewer";

  return `${name} (${role})`;
}
```

Follow-up: "What's the difference between `??` and `||`?"

`||` returns the right-hand side for any falsy value: `false`, `0`, `""`, `null`, `undefined`, `NaN`.

`??` returns the right-hand side only for `null` and `undefined`.

Use `??` when `0`, `""`, or `false` are valid values you want to keep. Use `||` when you want to fall back for any falsy value.

Q 4.8 What's the difference between type assertions (`as`) and the `satisfies` operator?

Difficulty: Mid **Asked at:** Google, Razorpay, Cred

Both are about telling TypeScript something about a type, but they work very differently.

Type assertion (`as`) — you override TypeScript's type

TYPESCRIPT Copy

```
// You're telling TypeScript: "Trust me, I know this is a string"
const input = document.getElementById("name") as HTMLInputElement;
input.value = "Ankit";  // Works — TypeScript treats it as HTMLInputElemen

// The danger: TypeScript believes you without checking
const data = fetchData() as User;
// If fetchData() actually returns null, TypeScript won't catch it
// You'll get a runtime error when you access data.name
```

Type assertions are an escape hatch. You're telling the compiler "I know better than you." Sometimes that's necessary (DOM elements, working with untyped libraries), but overusing `as` defeats the purpose of TypeScript.

`satisfies` operator — TypeScript checks the type BUT preserves the inferred type

This is the newer and generally safer option, added in TypeScript 4.9:

TYPESCRIPT Copy

```typescript
// Problem with type annotation:
const palette: Record<string, string> = {
  red: "#ff0000",
  green: "#00ff00",
  blue: "#0000ff",
};
// palette.red is typed as 'string' - TypeScript forgot the specific keys

// Problem with `as`:
const palette = {
  red: "#ff0000",
  green: "#00ff00",
  blue: "#0000ff",
} as Record<string, string>;
// Same problem - TypeScript forgets the specific keys
// Also: `as` wouldn't catch if you wrote blue: 123 (number instead of str

// Solution with `satisfies`:
const palette = {
  red: "#ff0000",
  green: "#00ff00",
  blue: "#0000ff",
} satisfies Record<string, string>;

// TypeScript CHECKS that all values are strings (catches errors)
// BUT preserves the literal types - palette.red is "#ff0000", not just st
// AND preserves the known keys - palette.purple would be an error
```

Here's a practical React example:

TYPESCRIPT Copy

```
  path: string;
  component: React.ComponentType;
  exact?: boolean;
};

// With annotation — loses literal path types
const routes: Route[] = [
  { path: "/", component: Home, exact: true },
  { path: "/about", component: About },
];
// routes[0].path is 'string' — not '/'

// With satisfies — validates AND preserves literal types
const routes = [
  { path: "/", component: Home, exact: true },
  { path: "/about", component: About },
] satisfies Route[];
// routes[0].path is '/' — exact literal type preserved
// If you misspelled 'component' as 'componet', TypeScript catches it
```

When to use which:

- `satisfies` — When you want TypeScript to validate that a value matches a type while keeping the narrower inferred type. This is the default choice in 2026.
- `as` — When working with DOM elements, third-party untyped data, or situations where TypeScript genuinely can't infer the correct type. Use sparingly.
- **Neither** — When a regular type annotation or generic does the job. Don't reach for `as` or `satisfies` if `const x: Type = value` works fine.

Common mistake: Using `as any` to silence TypeScript errors. This is almost always a sign that the types need fixing, not suppressing. Every `as any` is a potential runtime crash hiding behind a compile-time silence.

QUICK CHECK — no marks, just for you.

1. The lesson says you should write const items: string[] = [] instead of relying on inference. What type does TypeScript infer for an empty array literal when you leave the annotation

- never[]
- unknown[]
- any[]

Check answer

2. According to the lesson, which capability belongs to interface and NOT to a type alias?

- Defining a tuple type such as [number, number]
- Defining mapped and conditional types
- Declaration merging — declaring the same interface name twice merges the two declarations
- Defining a union of several types

Check answer

3. You have const STATUS = { Loading: "loading", Success: "success", Error: "error" } as const. Per the lesson, what does as const change compared with the same object written without it?

- The object compiles into a TypeScript enum with reverse mappings
- The property values stay typed as string but the object itself can no longer be reassigned
- Every property becomes readonly and each value is narrowed to its exact literal type instead of the wider string

Check answer

Previous
Performance and animation

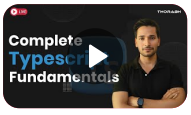
Next
Generics Advanced Types

LEARN MORE

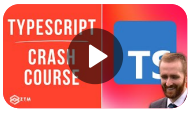
ARTICLES & DOCS

TypeScript Handbook
typescriptlang.org

VIDEOS



TypeScript fundamentals — full course
YouTube



TypeScript 101 crash course
YouTube



TypeScript — the basics
YouTube

Hand-picked references for this lesson.